

동기 vs 비동기, 블로킹 vs 논블로킹

□ 개념 및 정의

개념	정의
동기	요청된 작업이 끝나야 다음으로 넘어감 (결과를 직접 확인)
비동기	작업 요청 후 나중에 결과를 받아 처리함 (결과를 나중에 누군가 알려줌)
블로킹	요청된 작업이 끝나야 다음으로 넘어감 (작업이 끝날 때까지 기다림)
논블로킹	작업 요청 후 바로 다음 작업을 행함 (작업이 끝날 때까지 기다리지 않음)

□ 동기 vs 비동기, 블로킹 vs 논블로킹 상황 설정

- 커피를 마시고 싶은 A
- 직접 커피를 만들어 먹을 수 있는 무인 커피숍과 알바생이 만들어 주는 커피숍

□ 4가지 상황 비교

조합	A의 행동 시나리오
동기 + 블로킹	A가 직접 커피를 만들고 커피를 만드는 동안 다른 일은 하지 않음 (커피 완성 여부 직접 확인)
동기 + 논블로킹	A가 커피 만드는 기계의 버튼을 누르고 만들어지는 동안 다른 일 (커피 완성 여부 직접 확인)
비동기 + 블로킹	A는 알바생에게 커피를 주문하고 커피를 만드는 동안 다른 일은 하지 않음 (커피가 완성되면 알려줌)
비동기 + 논블로킹	A는 알바생에게 커피를 주문하고 만들어지는 동안 다른 일 (커피가 완성되면 알려줌)

- 동기 vs 비동기 : 결과를 직접 확인하는가? (알림)
- 블로킹 vs 논블로킹 : 결과가 나올 때까지 기다리는가? (대기)

□ 동기 + 블로킹

- 직접 처리하고 결과가 나올 때까지 기다리는 방식
- 보통 프로그램을 개발할 때 가장 흔히 사용되는 기본 구조
- 순차적이고 예측 가능하지만 오래 걸리는 작업이 있으면 프로그램 전체가 멈춤

<script>

```
console.log('A: 직접 커피 만들기 시작');
```

```
function makeCoffeeSync() {  
  const start = Date.now();  
  while (Date.now() - start < 3000) {} // 3초 멈춤  
  return '☕ 커피 완성';  
}
```

```
const coffee = makeCoffeeSync();  
console.log('A:', coffee);  
console.log('A: 이제 다른 일 시작');
```

</script>

A: 직접 커피 만들기 시작

A: ☕ 커피 완성

A: 이제 다른 일 시작

□ 동기 + 논블로킹

- 실제로는 거의 찾아보기 힘든 조합
- 의미있는 동작에서는 대부분 비동기 구조를 사용하므로 실용성 낮음
- ※ Promise를 사용하면 무조건 비동기이지만 동기처럼 보이도록 구성

<script>

```
console.log('A: 버튼 누름');
```

```
function pressButtonLikeSync() {  
  Promise.resolve('☕ 커피 자동 제작 중'); // 결과 무시  
  return '🔊 커피가 곧 나옵니다';  
}
```

```
const msg = pressButtonLikeSync(); // 즉시 반환  
console.log('A:', msg);  
console.log('A: 바로 자리로 가서 활동 시작');
```

</script>

A: 버튼 누름

A: 🔊 커피가 곧 나옵니다

A: 바로 자리로 가서 활동 시작

□ 비동기 + 블로킹

- 비동기 작업이지만 결과가 나올 때까지 기다리는 방식
- 외부 시스템 (서버, DB, 파일 등)과 통신할 때 자주 등장되는 구조
- `async`, `await` 등을 사용해서 블로킹이 될 부분을 제어

<script>

```
console.log('A: 알바생에게 커피 주문');
```

```
function orderCoffee() {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      resolve('☕ 커피 완성!');  
    }, 3000);  
  });  
}
```

```
const waitAtCounter = async () => {  
  const coffee = await orderCoffee(); // 기다림  
  console.log('A:', coffee);  
  console.log('A: 이제 다른 일 시작');  
};
```

```
waitAtCounter();
```

</script>

A: 알바생에게 커피 주문

A: ☕ 커피 완성!

A: 이제 다른 일 시작

□ 비동기 + 논블로킹

- 요청 후 다른 작업 수행
- `then`, `callback`, `event listener` 등으로 완료 시점을 나중에 처리

<script>

```
console.log('A: 커피 주문');
```

```
function orderCoffeeAsync() {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      resolve('☕ 커피 도착!');  
    }, 3000);  
  });  
}
```

```
orderCoffeeAsync().then((coffee) => {  
  console.log('A:', coffee);  
  console.log('A: 커피를 마신다');  
});
```

```
console.log('A: 기다리는 동안 책 읽고 유튜브 보기');
```

</script>

A: 커피 주문

A: 기다리는 동안 책 읽고 유튜브 보기

A: ☕ 커피 도착!

A: 커피를 마신다